

FrequenC Intern Project Brief

Simple NestJS + React CRUD Web App

Project duration: Maximum 2 days

Project repository name: frequenc-intern-project

Recommended app idea: Task Manager CRUD App

1. Project Overview

Build a simple full-stack web application using NestJS for the backend and React for the frontend. The application should allow users to manage tasks using basic CRUD operations. The goal is to evaluate fundamentals, code organization, API integration, form handling, validation, and clean UI implementation.

2. Core App Requirements

Feature	Required Behavior	Notes
Create Task	User can create a task with title, description, status, and due date.	Use a form with validation.
View Tasks	User can see all tasks in a list or table.	Show loading and empty states.
View Single Task	User can open/view details of one task.	Can be modal or separate page.
Update Task	User can edit task data.	Status must be editable.
Delete Task	User can delete a task.	Ask for confirmation before deleting.
Filter Tasks	User can filter by status.	Simple dropdown is enough.

3. Data Model

Use the following task model:

- id: string or number
- title: string, required
- description: string, optional
- status: pending | in_progress | completed
- dueDate: date, optional
- createdAt: date
- updatedAt: date

4. Backend Requirements - NestJS

- Create a NestJS REST API with a Tasks module, controller, service, and DTOs.
- Implement endpoints: GET /tasks, GET /tasks/:id, POST /tasks, PATCH /tasks/:id, DELETE /tasks/:id.
- Use validation with class-validator and class-transformer.
- Use either SQLite, PostgreSQL, MongoDB, or in-memory storage. For a 2-day task, SQLite or in-memory is acceptable.
- Return proper HTTP status codes and useful error messages.
- Keep the code organized and readable.

5. Frontend Requirements - React

- Create a React app using Vite and TypeScript.
- Create pages/components for task list, task form, task details, and task status filter.
- Connect to the NestJS API using fetch or Axios.
- Show loading states, error states, and empty states.
- Use clean component structure and reusable UI pieces where useful.
- Styling can be simple CSS, Tailwind, or any basic UI approach.

6. GitHub Rules

- Repository name must be: frequenc-intern-project.
- The repository must be public unless instructed otherwise.
- Use a clear README.md explaining setup, environment variables, and how to run the app.
- Use meaningful commit messages. Example: add task creation API, build task list UI, connect frontend to backend.
- Do not push node_modules, .env files, build folders, or generated local files.
- Use a .gitignore file for both frontend and backend.

7. Suggested Folder Structure

```
frequenc-intern-project/  
  backend/  
    src/  
    package.json  
    .env.example  
  frontend/  
    src/  
    package.json  
    .env.example  
  README.md
```

8. README Must Include

- Project name and short description.
- Tech stack used.
- Backend setup commands.
- Frontend setup commands.
- Environment variable examples.
- API endpoints list.
- Screenshots are optional but recommended.

9. Evaluation Criteria

Area	What Will Be Checked
Functionality	CRUD operations work correctly from frontend to backend.
Code Quality	Readable code, proper naming, organized files, no unnecessary complexity.
API Design	Correct REST endpoints, validation, and error handling.
Frontend Basics	Forms, lists, loading states, errors, and simple responsive layout.
Git Usage	Clean repository, meaningful commits, correct naming, useful README.

10. Optional Bonus Tasks

- Add search by task title.
- Add simple pagination.
- Add basic unit tests for the backend service.
- Add a simple dashboard showing task counts by status.
- Deploy the frontend and backend if time allows.

11. Submission Instructions

Submit the GitHub repository link after completing the project. The project will be reviewed, feedback will be provided, and the next task will be assigned after review.

Prepared for FrequenC internship evaluation.